

Lazy Loading in Entity Framework Core

In this article, I will discuss **Lazy Loading in Entity Framework (EF) Core** with Examples. Please read our previous article discussing [Eager Loading in Entity Framework Core](#) with Examples.

What is Lazy Loading in Entity Framework Core?

Lazy Loading in Entity Framework Core is a technique where related entities of a primary entity are only loaded from the database when they are accessed for the first time, rather than when the primary entity is initially retrieved. That means it is not fetched from the database until the related entity or collection is explicitly accessed for the first time. If the related entity or collection is not accessed, the additional data is not loaded from the database, saving both time and resources.

This allows the application to defer retrieving related data until it is actually needed. This improves performance by minimizing the initial amount of data retrieved, especially in scenarios where not all related data is needed upfront.

For example, if a Student entity has a navigation property to a related Branch entity, the Branch data will not be fetched from the database until we explicitly access the Student entity's branch property.

Example to understand Lazy Loading in Entity Framework Core:

Let us understand Lazy Loading in EF Core with an example. We will work with the same example we have worked on. Please look at the following student entity we created in our previous article.

```
public class Student
{
    0 references
    public int StudentId { get; set; }
    0 references
    public string FirstName { get; set; }
    0 references
    public string LastName { get; set; }
    0 references
    public string Gender { get; set; }
    0 references
    public DateTime DateOfBirth { get; set; }
    0 references
    public int BranchId { get; set; }
    0 references
    public Branch Branch { get; set; }
    0 references
    public Address Address { get; set; }
    0 references
    public ICollection<Course> Courses { get; set; }
}
```

↓ Related Entities of Student Entity

In the Student class, we have:

- A navigation property for the Address entity.

- A navigation property for the Branch entity.
- A collection navigation property for the Courses entities.

How Lazy Loading Works in EF Core:

In Lazy Loading, when we first query the Student entity, EF Core only loads the main entity data from the database (i.e., the Student table). The related data (from Address, Branch, and Courses tables) will not be loaded until we explicitly access the corresponding navigation properties. This allows us to defer the loading of related entities until they are actually needed.

Initial Query:

When we execute the following query, EF Core will only load the data from the Student table (not the related Address, Branch, or Courses data):

```
Student? student = context.Students.FirstOrDefault(std => std.StudentId == 1);
```

At this point, EF Core issues a single SQL query to fetch the Student data only. Related entities such as Address, Branch, and Courses have not yet been loaded.

Accessing the Address Navigation Property:

When we access the Address property for the first time:

```
Address? address = student?.Address;
```

EF Core will issue a new SQL query to the database to load the Address data related to the Student.

Accessing the Branch Navigation Property:

Similarly, when we access the Branch property:

```
Branch? branch = student?.Branch;
```

EF Core will generate another SQL query to fetch the Branch data related to the Student. The Branch entity is not loaded until the navigation property is accessed.

Accessing the Courses Collection Navigation Property:

Finally, if you access the Courses collection:

```
ICollection<Course> courses = student?.Courses;
```

EF Core will issue a separate SQL query to load all the Courses related to the Student. The Courses are lazily loaded only when the collection is accessed for the first time.

How Do We Implement Lazy Loading in EF Core?

EF Core does not support Lazy Loading by default. To implement Lazy Loading using EF Core, we need to follow the below steps:

- **Install the Required Package for Lazy Loading**
- **Enable Lazy Loading in the DbContext**
- **Mark Navigation Properties as virtual**

Step1: Install the Required Package for Lazy Loading:

We need to add the **Microsoft.EntityFrameworkCore.Proxies** package to our project. This package provides support for lazy loading of navigation properties. You can install this package using both NuGet Manager for Solution and by executing the following command in the Package Manager Console.

Install-Package Microsoft.EntityFrameworkCore.Proxies

Step2: Enable Lazy Loading in the DbContext:

Once the package is installed, we need to enable lazy loading in our DbContext class by overriding the OnConfiguring method. This needs to be done by calling the **UseLazyLoadingProxies()** method on the DbContextOptionsBuilder object. So, modify the EFCoreDbContext class as follows:

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Logging;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // Enable Logging
            optionsBuilder.LogTo(Console.WriteLine, LogLevel.Information);
            //Enabling Lazy Loading
            optionsBuilder.UseLazyLoadingProxies();
            //Connection String
            optionsBuilder.UseSqlServer("Server=LAPTOP-6P5NK25R\\SQLSERVER2022DEV;Database=StudentDB;Trusted_Connection=True;TrustServerCertificate=True");
            // DbSet properties represent the tables in the database.
            // Each DbSet corresponds to a table, and the type parameter corresponds
            mapped to that table.
            public DbSet<Student> Students { get; set; }
            public DbSet<Teacher> Teachers { get; set; }
            public DbSet<Branch> Branches { get; set; }
            public DbSet<Address> Addresses { get; set; }
            public DbSet<Subject> Subjects { get; set; }
            public DbSet<Course> Courses { get; set; }
        }
    }
}
```

Step3: Mark Navigation Properties as virtual:

For Lazy Loading to work with Entity Framework Core, we need to mark the all-navigation properties of all entities as virtual. EF Core will create proxy classes to override these virtual properties and trigger the lazy loading mechanism when they are accessed. So, please modify all the entities as follows. Here, we are marking all the navigation properties as virtual.

Student Entity:

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Student
    {
        public int StudentId { get; set; }
        public string FirstName { get; set; }
    }
}
```

```

        public string LastName { get; set; }
        public string Gender { get; set; }
        public DateTime DateOfBirth { get; set; }
        public int BranchId { get; set; }
        public virtual Branch Branch { get; set; } //Marking the Property as
Virtual to Support Lazy Loading
        public virtual Address Address { get; set; } //Marking the Property
as Virtual to Support Lazy Loading
        public virtual ICollection<Course> Courses { get; set; } //Marking
the Property as Virtual to Support Lazy Loading
    }
}

```

Course Entity:

```

namespace EFCoreCodeFirstDemo.Entities
{
    public class Course
    {
        public int CourseId { get; set; }
        public string Name { get; set; }
        public string Description { get; set; }
        public string Fees { get; set; }
        public virtual ICollection<Student> Students { get; set; } //Marking
the Property as Virtual to Support Lazy Loading
        public virtual ICollection<Subject> Subjects { get; set; } //Marking
the Property as Virtual to Support Lazy Loading
    }
}

```

Teacher Entity:

```

namespace EFCoreCodeFirstDemo.Entities
{
    public class Teacher
    {
        public int TeacherId { get; set; }
        public string TeacherName { get; set; }
        public int BranchId { get; set; }
        public virtual Branch Branch { get; set; }
        public virtual Address Address { get; set; }
        public virtual ICollection<Subject> Subjects { get; set; }
    }
}

```

Subject Entity:

```

namespace EFCoreCodeFirstDemo.Entities
{
    public class Subject

```

```

{
    public int SubjectId { get; set; }
    public string SubjectName { get; set; }
    public string Description { get; set; }
    public virtual ICollection<Teacher> Teachers { get; set; }
    public virtual ICollection<Course> Courses { get; set; }
}
}

```

Address Entity:

```

namespace EFCoreCodeFirstDemo.Entities
{
    public class Address
    {
        public int AddressId { get; set; }
        public string Street { get; set; }
        public string City { get; set; }
        public string State { get; set; }
        public string PostalCode { get; set; }
        public int? StudentId { get; set; }
        public virtual Student Student { get; set; } //Marking the Property
as Virtual to Support Lazy Loading
        public int? TeacherId { get; set; }
        public virtual Teacher Teacher { get; set; } //Marking the Property
as Virtual to Support Lazy Loading
    }
}

```

Branch Entity:

```

namespace EFCoreCodeFirstDemo.Entities
{
    public class Branch
    {
        public int BranchId { get; set; }
        public string BranchLocation { get; set; }
        public string? BranchPhoneNumber { get; set; }
        public string? BranchEmail { get; set; }
        public virtual ICollection<Student> Students { get; set; }
        public virtual ICollection<Teacher> Teachers { get; set; }
    }
}

```

Generating Migration:

Whenever we add or update domain classes, we need to sync the database with the model using **Add-Migration** and **Update-Database** commands using Package Manager Console. So, please execute the **Add-Migration** and **Update-Database** commands as follows:

```
Package Manager Console
Package source: All [v] [g] Default project: EFCoreCodeFirstDemo [v]
PM> Add-Migration LazyLoadingSuuport
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
```

Next, update the database by executing the **Update-Database** command in the command prompt, as shown in the below image.

```
Package Manager Console
Package source: All [v] [g] Default project: EFCoreCodeFirstDemo [v]
PM> Update-Database
Build started...
Build succeeded.
```

Example to Understand Lazy Loading in EF Core:

Now, modify the Program class as follows to use Lazy Loading in Entity Framework Core. The following example is self-explained, so please read the comment lines for a better understanding. Here, we are Lazy Loading the related Branch, Address, and Courses entities of the Student entity.

```
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            using (var context = new EFCoreDbContext())
            {
                try
                {
                    // Lazy Loading Example
                    Console.WriteLine("Lazy Loading Student and related
data\n");

                    // Load a student (only student data is loaded initially)
                    var student = context.Students.FirstOrDefault(s =>
s.StudentId == 1);

                    // Display basic student information
                    Console.WriteLine($"Student Id: {student?.StudentId},
Name: {student?.FirstName} {student?.LastName}, Gender: {student?.Gender}
\n");

                    // Accessing the Branch property triggers lazy loading
                    // EF Core will issue a SQL query to load the related
Branch
```

```

        if (student != null)
        {
            Console.WriteLine($"Branch Location:
{student.Branch?.BranchLocation}, Email: {student.Branch?.BranchEmail},
Phone: {student.Branch?.BranchPhoneNumber} \n");
            // Accessing the Address property triggers lazy
loading
            // EF Core will issue a SQL query to load the related
Address
            Console.WriteLine($"Address:
{student.Address?.Street}, {student.Address?.City}, {student.Address?.State},
Pin: {student.Address?.PostalCode} \n");
            // Accessing the Courses collection triggers lazy
loading
            // EF Core will issue a SQL query to load the related
Courses and their related Subjects
            //foreach (var course in student.Courses)
            //{
            //    Console.WriteLine($"Course Enrolled:
{course.Name}");
            //You can also access the Subjects of each as
follows
            //foreach (var subject in course.Subjects)
            //{
            //    Console.WriteLine($"    Subject:
{subject.SubjectName}");
            //}
            //}
        }
    }
    catch (Exception ex)
    {
        // Handle any errors that occur during data retrieval
        Console.WriteLine($"An error occurred: {ex.Message}");
    }
}
// Final Output
Console.WriteLine("\nLazy loading of related entities
completed.");
}
}
}
}

```

Output:

```

Lazy Loading Student and related data

info: 08-09-2024 10:58:31.576 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (10ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT TOP(1) [s].[StudentId], [s].[BranchId], [s].[DateOfBirth], [s].[FirstName], [s].[Gender], [s].[LastName]
      FROM [Students] AS [s]
      WHERE [s].[StudentId] = 1

Student Id: 1, Name: John Doe, Gender: Male ⬆ SELECT Statement to Load Data from Students table

info: 08-09-2024 10:58:31.667 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (16ms) [Parameters=[@__p_0='?' (DbType = Int32)], CommandType='Text', CommandTimeout='30']
      SELECT TOP(1) [b].[BranchId], [b].[BranchEmail], [b].[BranchLocation], [b].[BranchPhoneNumber]
      FROM [Branches] AS [b]
      WHERE [b].[BranchId] = @__p_0 ⬆ Another SELECT Statement to Load Data from Branches table

Branch Location: BBSR, Email: BBSR@dotnettutorials.net, Phone: 212-555-1234

info: 08-09-2024 10:58:31.703 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (1ms) [Parameters=[@__p_0='?' (DbType = Int32)], CommandType='Text', CommandTimeout='30']
      SELECT TOP(1) [a].[AddressId], [a].[City], [a].[PostalCode], [a].[State], [a].[Street], [a].[StudentId], [a].[TeacherId]
      FROM [Addresses] AS [a]
      WHERE [a].[StudentId] = @__p_0 ⬆ Another SELECT Statement to Load Data from Addresses table

Address: 123 Main St, New York, NY, Pin: 10001

```

As you can see in the above output, it is issuing three different SQL SELECT Statements to load the data from the database. That means it uses lazy loading to load the related data. Further, you can see that it is not loading the Courses table data, as we have commented on the statement that accesses the Courses navigation property.

Note: With Lazy Loading enabled in EF Core, EF Core will automatically load the data from the database when we access a related entity for the first time. The loaded data is then stored in the context's change tracker (in memory). If we reaccess the same related entity within the same DbContext instance, the data is fetched from in-memory rather than querying the database again. This avoids redundant database calls and improves performance for subsequent access to the same entity.

Can we use both Eager Loading and Lazy Loading using EF Core?

We can use both Eager Loading and Lazy Loading together in EF Core. They can coexist in a scenario where:

- Some related entities are loaded eagerly when querying the primary entity.
- Other related entities are loaded lazily when they are accessed later.

Let's understand this with one example. We want to use Eager Loading for the Branch entity and Lazy Loading for the Address entity. To better understand this, please modify the Program class as follows:

```

using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            using (var context = new EFCoreDbContext())
            {
                try
                {
                    // Eager Loading Example for Branch, Lazy Loading for
                    Address

```



```

        Console.WriteLine("Eager Loading Branch, Lazy Loading
Address\n");

        // Load a student and related Branch using Eager Loading
        var student = context.Students
            .Include(s => s.Branch) // Eagerly
load the Branch entity
            .FirstOrDefault(s => s.StudentId ==
1);

        // Display basic student information
        if (student != null)
        {
            Console.WriteLine($"Student Id:
{student.StudentId}, Name: {student.FirstName} {student.LastName}, Gender:
{student.Gender}");

            // Check if Branch is null
            if (student.Branch != null)
            {
                Console.WriteLine($"Branch Location:
{student.Branch.BranchLocation}, Email: {student.Branch.BranchEmail}, Phone:
{student.Branch.BranchPhoneNumber}\n");
            }
            else
            {
                Console.WriteLine("Branch data not
available.\n");
            }
        }
        // Accessing the Address property triggers lazy
loading
        // EF Core will issue a SQL query to load the related
Address
        if (student.Address != null)
        {
            Console.WriteLine($"Address:
{student.Address.Street}, {student.Address.City}, {student.Address.State},
Pin: {student.Address.PostalCode}");
        }
        else
        {
            Console.WriteLine("\nAddress data not
available.");
        }
    }
    else
    {
        Console.WriteLine("Student data not found.");
    }
}
catch (Exception ex)
{
    // Handle any errors that occur during data retrieval
    Console.WriteLine($"An error occurred: {ex.Message}");
}

```

```

    }
}
// Final Output
Console.WriteLine("\nEager loading of Branch and lazy loading of
Address completed.");
}
}
}
}

```

In the above example, Eager Loading is done by using the Include method to load the Branch entity when the Student entity is queried. Lazy loading is enabled for our application, and when we access the Address entity for the first time, it will issue a select query to the database to fetch the data. So, run the application, and you should see the following output:

```

info: 08-09-2024 11:18:00.063 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (22ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT TOP(1) [s].[StudentId], [s].[BranchId], [s].[DateOfBirth], [s].[FirstName], [s].[Gender], [s].[LastName],
      [b].[BranchId], [b].[BranchEmail], [b].[BranchLocation], [b].[BranchPhoneNumber]
      FROM [Students] AS [s]
      INNER JOIN [Branches] AS [b] ON [s].[BranchId] = [b].[BranchId]
      WHERE [s].[StudentId] = 1
      ↑ Eager Loading using JOIN

Student Id: 1, Name: John Doe, Gender: Male
Branch Location: BBSR, Email: BBSR@dotnettutorials.net, Phone: 212-555-1234

info: 08-09-2024 11:18:00.167 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (18ms) [Parameters=[@__p_0=?' (DbType = Int32)], CommandType='Text', CommandTimeout='30']
      SELECT TOP(1) [a].[AddressId], [a].[City], [a].[PostalCode], [a].[State], [a].[Street], [a].[StudentId], [a].[Tea
      cherId]
      FROM [Addresses] AS [a]
      WHERE [a].[StudentId] = @__p_0
      ↑ Lazy Loading using a Separate SELECT SQL Statement

Address: 123 Main St, New York, NY, Pin: 10001

Eager loading of Branch and lazy loading of Address completed.

```

How Do We Disable Lazy Loading in EF Core?

In Entity Framework Core (EF Core), we have several options to disable lazy loading, depending on how we want to control the behavior. Let's proceed to understand these options.

Do Not Use Lazy Loading Proxies

If you haven't explicitly enabled lazy loading by adding lazy loading proxies (via **UseLazyLoadingProxies()**), lazy loading will not be used. This is the simplest way to ensure lazy loading is disabled. Please remove the following line from your DbContext configuration to disable Lazy Loading:

```

// Remove this line to disable lazy loading proxies
optionsBuilder.UseLazyLoadingProxies();

```

Once you remove or comment on the above code in the EFCoreDbContext class, run the application, and you will see the following output. You can see Eager loading, i.e., loading the Branch entity, is working, but Lazy loading, i.e., Loading the Address entity, is not working.

Eager Loading Branch, Lazy Loading Address

```
info: 08-09-2024 11:29:03.375 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (27ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT TOP(1) [s].[StudentId], [s].[BranchId], [s].[DateOfBirth], [s].[FirstName], [s].[LastName],
[b].[BranchId], [b].[BranchEmail], [b].[BranchLocation], [b].[BranchPhoneNumber]
      FROM [Students] AS [s]
      INNER JOIN [Branches] AS [b] ON [s].[BranchId] = [b].[BranchId]
      WHERE [s].[StudentId] = 1
```

Student Id: 1, Name: John Doe, Gender: Male
Branch Location: BBSR, Email: BBSR@dotnettutorials.net, Phone: 212-555-1234

Address data not available.

← Lazy Loading not working

Disable Lazy Loading Globally in DbContext

We can also disable lazy loading globally by setting the **LazyLoadingEnabled** option to false in the DbContext constructor. So, for a better understanding, please modify the EFCoreDbContext class as follows. The **this.ChangeTracker.LazyLoadingEnabled = false** statement disable Lazy loading.

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Logging;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        //Constructor calling the Base DbContext Class Constructor
        public EFCoreDbContext() : base()
        {
            //Disabling Lazy Loading
            this.ChangeTracker.LazyLoadingEnabled = false;
        }
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // Enable Logging
            optionsBuilder.LogTo(Console.WriteLine, LogLevel.Information);
            //Connection String
            optionsBuilder.UseSqlServer("Server=LAPTOP-6P5NK25R\\SQLSERVER2022DEV;Database=StudentDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        // DbSet properties represent the tables in the database.
        // Each DbSet corresponds to a table, and the type parameter corresponds
        mapped to that table.
        public DbSet<Student> Students { get; set; }
        public DbSet<Teacher> Teachers { get; set; }
        public DbSet<Branch> Branches { get; set; }
        public DbSet<Address> Addresses { get; set; }
        public DbSet<Subject> Subjects { get; set; }
        public DbSet<Course> Courses { get; set; }
    }
}
```

Now, with these changes in place, run the application. You will see the following output as expected: Lazy loading is not working as expected for the Address entity.

Eager Loading Branch, Lazy Loading Address

```
info: 08-09-2024 11:29:03.375 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (27ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT TOP(1) [s].[StudentId], [s].[BranchId], [s].[DateOfBirth], [s].[FirstName], [s].[Gender], [s].[LastName],
      [b].[BranchId], [b].[BranchEmail], [b].[BranchLocation], [b].[BranchPhoneNumber]
      FROM [Students] AS [s]
      INNER JOIN [Branches] AS [b] ON [s].[BranchId] = [b].[BranchId]
      WHERE [s].[StudentId] = 1
```

Student Id: 1, Name: John Doe, Gender: Male
Branch Location: BBSR, Email: BBSR@dotnettutorials.net, Phone: 212-555-1234

Address data not available.

↑ Eager Loading working

← Lazy Loading not working

Note: The default value of the **LazyLoadingEnabled** property is true. This is why, even though we have not set this property value to true, Lazy Loading is working in our application when we use **UseLazyLoadingProxies()**.

Programmatically Enabling and Disabling Lazy Loading:

If we want, we can enable and disable lazy loading while accessing the data. For a better understanding, please have a look at the following example. In the below example, after fetching the Student data, we disable Lazy Loading by using the statement **context.ChangeTracker.LazyLoadingEnabled = false;** hence, it will not load the Branch data by issuing a SELECT SQL Statement.

Then, we enabled lazy loading using the **context.ChangeTracker.LazyLoadingEnabled = true;** statement. Then, we access the Address navigation property. This time, as Lazy Loading is enabled, it will issue a separate SELECT SQL Statement and load the Student Address data.

```
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            using (var context = new EFCoreDbContext())
            {
                try
                {
                    // Lazy Loading Example
                    Console.WriteLine("Lazy Loading Student and related
data\n");

                    // Load a student (only student data is loaded initially)
                    var student = context.Students.FirstOrDefault(s =>
s.StudentId == 1);

                    // Display basic student information
                    if (student != null)
                    {
                        Console.WriteLine($"Student Id:
{student.StudentId}, Name: {student.FirstName} {student.LastName}, Gender:
{student.Gender} \n");

                        //Disabling Lazy Loading Here
                        context.ChangeTracker.LazyLoadingEnabled = false;
```

```

        // Check if Branch is null before accessing its
properties
        if (student.Branch != null)
        {
            Console.WriteLine($"Branch Location:
{student.Branch.BranchLocation}, Email: {student.Branch.BranchEmail}, Phone:
{student.Branch.BranchPhoneNumber} \n");
        }
        else
        {
            Console.WriteLine("\nBranch data not
available.\n");
        }
        //Enabling Lazy Loading Here
        context.ChangeTracker.LazyLoadingEnabled = true;
        // Check if Address is null before accessing its
properties
        if (student.Address != null)
        {
            Console.WriteLine($"Address:
{student.Address.Street}, {student.Address.City}, {student.Address.State},
Pin: {student.Address.PostalCode} \n");
        }
        else
        {
            Console.WriteLine("\nAddress data not
available.\n");
        }
    }
    else
    {
        Console.WriteLine("Student data not found.");
    }
}
catch (Exception ex)
{
    // Handle any errors that occur during data retrieval
    Console.WriteLine($"An error occurred: {ex.Message}");
}
}
// Final Output
Console.WriteLine("\nLazy loading of related entities
completed.");
}
}
}

```

Modifying the DbContext:

Before running the application, please modify the **EFCoreDbContext** class as follows. Here, we are enabling the Lazy Loading for our application.

```

using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Logging;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // Enable Logging
            optionsBuilder.LogTo(Console.WriteLine, LogLevel.Information);
            //Enable Lazy Loading
            optionsBuilder.UseLazyLoadingProxies();
            //Connection String
            optionsBuilder.UseSqlServer("Server=LAPTOP-6P5NK25R\\SQLSERVER2022DEV;Database=StudentDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        // DbSet properties represent the tables in the database.
        // Each DbSet corresponds to a table, and the type parameter corresponds to that table.
        public DbSet<Student> Students { get; set; }
        public DbSet<Teacher> Teachers { get; set; }
        public DbSet<Branch> Branches { get; set; }
        public DbSet<Address> Addresses { get; set; }
        public DbSet<Subject> Subjects { get; set; }
        public DbSet<Course> Courses { get; set; }
    }
}

```

With the above changes in place, run the application and see the output. You will see Branch data will not be loaded, but Address data will be loaded by issuing a SELECT SQL Statement, as shown in the below image:

Lazy Loading Student and related data

```

info: 08-09-2024 11:45:46.924 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (18ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT TOP(1) [s].[StudentId], [s].[BranchId], [s].[DateOfBirth], [s].[FirstName], [s].[Gender], [s].[LastName]
      FROM [Students] AS [s]
      WHERE [s].[StudentId] = 1

```

Student Id: 1, Name: John Doe, Gender: Male

Branch data not available.

```

info: 08-09-2024 11:45:47.022 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (19ms) [Parameters=[@__p_0='?' (DbType = Int32)], CommandType='Text', CommandTimeout='30']
      SELECT TOP(1) [a].[AddressId], [a].[City], [a].[PostalCode], [a].[State], [a].[Street], [a].[StudentId], [a].[TeacherId]
      FROM [Addresses] AS [a]
      WHERE [a].[StudentId] = @__p_0

```

Address: 123 Main St, New York, NY, Pin: 10001

How Does Lazy Loading Work Internally with EF Core?

When we enable lazy loading in EF Core by calling `UseLazyLoadingProxies()` in the `OnConfiguring` method of the `DbContext` class, EF Core automatically generates proxy objects for our entities with virtual navigation properties. These proxy objects override these virtual navigation properties to dynamically load the related entities when accessed for the first time.

EF Core dynamically generates these proxy classes at runtime. These proxy classes inherit from our original entity classes and override virtual navigation properties (like Branch, Address, and Courses). These override navigation properties contain special logic to handle the automatic loading of related data from the database whenever the properties are accessed.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Student
    {
        public int StudentId { get; set; }
        public string FirstName { get; set; }
        public string LastName { get; set; }
        public string Gender { get; set; }
        public DateTime DateOfBirth { get; set; }
        public int BranchId { get; set; }
        // Navigation properties must be virtual to support lazy loading
        public virtual Branch Branch { get; set; }
        public virtual Address Address { get; set; }
        public virtual ICollection<Course> Courses { get; set; }
    }
}
```

How EF Core Creates Proxy Objects:

The EF Core generates a proxy class at runtime that overrides the navigation properties, such as Branch. The proxy class is responsible for checking if the Branch entity is already loaded, and if not, it triggers a database query to load it. The following is a conceptual view of what happens under the hood:

```
public class StudentProxy : Student
{
    private Branch _branch;
    // Overriding the Branch property to implement lazy loading
    public override Branch Branch
    {
        get
        {
            // Check if the related entity (Branch) has been loaded
            if (_branch == null)
            {
                // If not loaded, issue a query to load the Branch entity
                from the database
                _branch = LoadBranchFromDatabase();
            }
            // Return the Branch entity
            return _branch;
        }
        set
        {
            _branch = value;
        }
    }
}
```

```

private Branch LoadBranchFromDatabase()
{
    // Logic to load the Branch entity from the database
    // This part is managed by EF Core and typically involves issuing a
SELECT query
    return EFCoreLazyLoadHelper.LoadRelatedEntity<Branch>
(this.StudentId);
}
}

```

What is the Difference Between Eager Loading and Lazy Loading?

Eager Loading:

- With eager loading, all the necessary data, including both the main entity and its related entities, is retrieved immediately in a single query when the primary entity is accessed. EF Core uses SQL joins (JOIN) to load related data along with the main entity.
- **Advantage:** Since all data is retrieved in a single query, there is only one round trip to the database, which can be more efficient when the related data is immediately needed. It reduces the load on the database.
- **Disadvantage:** If you don't need all of the related data, eager loading may result in over-fetching data, leading to unnecessary memory consumption and potentially slowing down the query if the data set is large.

Lazy Loading:

- With lazy loading, only the main entity's data is retrieved at first. Related data (e.g., Address, Courses, Branch) is not loaded until you explicitly access the navigation property. When you access a related entity for the first time, EF Core issues a separate query to load that related data.
- **Advantage:** You only load the data when you need it. This avoids over-fetching data and can improve performance when related data is not required upfront.
- **Disadvantage:** Lazy loading can result in multiple round trips to the database (the N+1 query problem). This can negatively impact performance, especially when dealing with large datasets or multiple related entities. It increases the load on the database server.

Which is Better: Eager Loading or Lazy Loading in EF Core?

Without looking at the application requirements and what we are trying to achieve, we cannot simply say one approach is better than the other. Lazy Loading and Eager Loading have their advantages and disadvantages, and we have seen clear performance differences between these approaches to achieving the same task.

When to Use Eager Loading vs. Lazy Loading

Choosing between eager loading and lazy loading should be based on the requirements of your application and the specific scenarios where data is accessed. You should evaluate the following considerations when deciding:

Use Eager Loading when:

- You know in advance that the related data is always required.
- You want to avoid multiple round trips to the database and ensure all necessary data is fetched in a single query.
- You are dealing with batch processing or report generation, where all related data is needed upfront.

Use Lazy Loading when:

- You don't always need the related data and want to defer loading until the data is explicitly accessed.
- You want to minimize the initial query size and only load related data as needed.
- You are working with user-triggered actions, loading related data based on user interaction (e.g., expanding a section in the UI to view related information).

So, the conclusion is that, based on your application requirements, you need to choose whether to use Lazy Loading or Eager Loading in Entity Framework Core.

In the next article, I will discuss [Explicit Loading in Entity Framework Core](#) with Examples. In this article, I try to explain Lazy Loading in Entity Framework Core with Examples. I hope you enjoyed this article. Please provide your valuable feedback and suggestions for this article.